Experiment 7 :

```
#include <stdio.h>

int findvalue(char c)
{
   return c - '0';
}

int main()
{
   int noalpha, nostate, notrans;
   int eclos[10][20] = {0};
   int i, j, k, x;
   int count, state, next, found;
   char alpha[5], trans[30][3];

   printf("Enter the number of
alphabets?\n");
   scanf("%d", &noalpha);

   printf("NOTE:- [ use letter e as
epsilon ]\n");

   printf("\nEnter alphabets?\n");
   for(i = 0; i < noalpha; i++)
   {
      getchar();
      alpha[i] = getchar();
   }

   printf("\nEnter the number of
states?\n");
   scanf("%d", &nostate);

   printf("\nEnter no of
transition?\n");
   scanf("%d", &notrans);

   printf("\nEnter transition?\n");

   for(i = 0; i < notrans; i++)
   {
      getchar();
      trans[i][0] = getchar();
      getchar();
      trans[i][1] = getchar();
      getchar();
      trans[i][2] = getchar();
   }

   printf("\ne-closure of states\n");

   for(i = 1; i <= nostate; i++)
   {
      count = 0;

      eclos[i][count++] = i;

      for(j = 0; j < count; j++)
      {
         state = eclos[i][j];

         for(k = 0; k < notrans; k++)
         {
            if(findvalue(trans[k][0])
== state &&
               trans[k][1] == 'e')
            {
               next =
findvalue(trans[k][2]);

               found = 0;

               for(x = 0; x < count;
x++)
               {
                  if(eclos[i][x] == next)
                  {
                     found = 1;
                     break;
                  }
               }

               if(found == 0)
               {
                  eclos[i][count] = next;
                  count++;
               }
            }
         }
      }
   }

   for(i = 1; i <= nostate; i++)
   {
      printf("e-closure of q%d - {", i);

      for(j = 0; j < 20 && eclos[i][j]
!= 0; j++)
      {
         printf("q%d", eclos[i][j]);

         if(eclos[i][j + 1] != 0)
            printf(",");
      }

      printf("}\n");
   }

   return 0;
}
```

OUTPUT :

```
Enter the number of alphabets?
3
NOTE:- [ use letter e as epsilon ]

Enter alphabets?
0
1
e

Enter the number of states?
4

Enter no of transition?
4

Enter transition?
1 e 2
1 0 3
2 e 4
3 1 4

e-closure of states
e-closure of q1 - {q1,q2,q4}
e-closure of q2 - {q2,q4}
e-closure of q3 - {q3}
e-closure of q4 - {q4}
```

Experiment 8 :

```
#include <stdio.h>

int n, m;
char alphabet[10];
int trans[10][10][10];
int dfa[20][10];
int dfa_count = 0;
exists */
int findstate(int state[]){
   int i, j;

   for(i = 0; i < dfa_count; i++){
      for(j = 1; j <= n; j++){
         if(dfa[i][j] != state[j])
            break;
      }

      if(j > n)
         return i;
   }

   return -1;
}

void addstate(int state[]){
   int i;

   if(findstate(state) == -1) {
      for(i = 1; i <= n; i++)
         dfa[dfa_count][i] = state[i];

      dfa_count++;
   }
}

void printstate(int state[]){
   int i, empty = 1;

   printf("{");

   for(i = 1; i <= n; i++){
      if(state[i])  {
         printf("q%d", i);
         empty = 0;
      }
   }

   if(empty)
      printf("NULL");

   printf("}");
}

int main(){
   int i, j, k;
   int start, nofinal, notrans;
   int final[10];

   int state[10], newstate[10];
   int current, from, to;
   char symbol;

   printf("Enter number of alphabets:
");
   scanf("%d", &m);

   printf("Enter alphabets:\n");
   for(i = 0; i < m; i++)
      scanf(" %c", &alphabet[i]);

   printf("Enter number of states: ");
   scanf("%d", &n);

   printf("Enter start state: ");
   scanf("%d", &start);

   printf("Enter number of final
states: ");
   scanf("%d", &nofinal);

   printf("Enter final states:\n");
   for(i = 0; i < nofinal; i++)
      scanf("%d", &final[i]);

   printf("Enter number of
transitions: ");
   scanf("%d", &notrans);

   printf("Enter transitions:\n");

   for(i = 0; i < notrans; i++){
      scanf("%d %c %d", &from,
&symbol, &to);

      for(j = 0; j < m; j++){
         if(alphabet[j] == symbol)
            trans[from][j][to] = 1;
      }
   }

   for(i = 1; i <= n; i++)
      state[i] = 0;

   state[start] = 1;
   addstate(state);
   current = 0;

   while(current < dfa_count){
      for(i = 0; i < m; i++){
         for(j = 1; j <= n; j++)
            newstate[j] = 0;

         for(j = 1; j <= n; j++){
            if(dfa[current][j]){
               for(k = 1; k <= n; k++){
                  if(trans[j][i][k])
                     newstate[k] = 1;
               }
            }
         }

         printf("\n");
         printstate(dfa[current]);
         printf(" --%c--> ",
alphabet[i]);
         printstate(newstate);

         addstate(newstate);
      }

      current++;
   }
   printf("\n\nStates of DFA:\n");

   for(i = 0; i < dfa_count; i++){
      printf("D%d = ", i);
      printstate(dfa[i]);

      for(j = 1; j <= n; j++){
         if(dfa[i][j]){
            for(k = 0; k < nofinal; k++){
               if(j == final[k]){
                  printf("  (Final)");
                  break;
               }
            }
         }
      }
```

```
        printf("\n");
    }

    printf("\nStart state: ");
    printstate(dfa[0]);

    return 0;
}
```

OUTPUT :

```
Enter number of alphabets: 2
Enter alphabets:
0
1
Enter number of states: 4
Enter start state: 1
Enter number of final states: 1
Enter final states:
3
Enter number of transitions: 5
Enter transitions:
1 0 1
1 0 2
1 1 1
2 1 3
3 0 3

{q1} --0--> {q1q2}
{q1} --1--> {q1}
{q1q2} --0--> {q1q2}
{q1q2} --1--> {q1q3}
{q1q3} --0--> {q1q2q3}
{q1q3} --1--> {q1}
{q1q2q3} --0--> {q1q2q3}
{q1q2q3} --1--> {q1q3}

States of DFA:
D0 = {q1}
D1 = {q1q2}
D2 = {q1q3}  (Final)
D3 = {q1q2q3}  (Final)

Start state: {q1}
```

Experiment 9 :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char p[20][20], first[26][20],
follow[26][20];
int n;

int present(char s[], char c){
    return strchr(s, c) != NULL;
}

void add(char s[], char c){
    if(!present(s, c)){
        int l = strlen(s);
        s[l] = c;
        s[l + 1] = '\0';
    }
}

void FIRST(char c, char r[]){
    int i, j, k;
    char t[20];

    if(!isupper(c)){
        add(r, c);
        return;
    }

    for(i = 0; i < n; i++)  {
        if(p[i][0] == c){
            if(p[i][3] == 'e')
                add(r, 'e');
            else{
                for(j = 3; p[i][j]; j++){
                    strcpy(t, "");
                    FIRST(p[i][j], t);

                    for(k = 0; t[k]; k++)
                        if(t[k] != 'e')
                            add(r, t[k]);

                    if(!present(t, 'e'))
                        break;

                    if(!p[i][j + 1])
                        add(r, 'e');
                }
            }
        }
    }
}

void FOLLOW(char c, char r[]){
    int i, j, k;
    char t[20], u[20];

    if(c == p[0][0])
        add(r, '$');

    for(i = 0; i < n; i++){
        for(j = 3; p[i][j]; j++){
            if(p[i][j] == c){
                if(p[i][j + 1]){
                    strcpy(t, "");
                    FIRST(p[i][j + 1], t);

                    for(k = 0; t[k]; k++)
                        if(t[k] != 'e')
                            add(r, t[k]);

                    if(present(t, 'e')){
                        strcpy(u, "");
                        FOLLOW(p[i][0], u);

                        for(k = 0; u[k]; k++)
                            add(r, u[k]);
                    }
                }
                else if(p[i][0] != c){
                    strcpy(u, "");
                    FOLLOW(p[i][0], u);

                    for(k = 0; u[k]; k++)
                        add(r, u[k]);
                }
            }
        }
    }
}

int main(){
    int i, j;
    char c;

    printf("Enter number of
productions: ");
    scanf("%d", &n);
```

```
    printf("Enter productions (use e
for epsilon):\n");

    for(i = 0; i < n; i++)
        scanf("%s", p[i]);

    printf("\nFIRST:\n");

    for(i = 0; i < n; i++){
        c = p[i][0];

        if(!first[c - 'A'][0])
            FIRST(c, first[c - 'A']);

        if(i == 0 || p[i - 1][0] != c)
        {
            printf("FIRST(%c) = { ", c);

            for(j = 0; first[c - 'A'][j]; j++)
                printf("%c ", first[c -
'A'][j]);

            printf("}\n");
        }
    }

    printf("\nFOLLOW:\n");

    for(i = 0; i < n; i++){
        c = p[i][0];

        if(!follow[c - 'A'][0])
            FOLLOW(c, follow[c - 'A']);

        if(i == 0 || p[i - 1][0] != c)
        {
            printf("FOLLOW(%c) = { ",
c);

            for(j = 0; follow[c - 'A'][j];
j++)
                printf("%c ", follow[c -
'A'][j]);

            printf("}\n");
        }
    }
    return 0;
}
```

OUTPUT :

```
Enter number of productions: 8
Enter productions (use e for epsilon):
E->TX
X->+TX
X->e
T->FY
Y->*FY
Y->e
F->(E)
F->i

FIRST:
FIRST(E) = { ( i }
FIRST(X) = { + e }
FIRST(T) = { ( i }
FIRST(Y) = { * e }
FIRST(F) = { ( i }

FOLLOW:
FOLLOW(E) = { $ ) }
FOLLOW(X) = { $ ) }
FOLLOW(T) = { + $ ) }
FOLLOW(Y) = { + $ ) }
FOLLOW(F) = { * + $ ) }
```

OUTPUT :

```
Enter expression: a+a*a
Accepted

Enter expression: (a+a)*a
Accepted

Enter expression: a+*a
Rejected

Enter expression: (a+a
Rejected

Enter expression: a*(a+a)
Accepted
```

Experiment 10 :

```
#include <stdio.h>
#include <string.h>
char s[20];
int i = 0, error = 0;
void E();
void T();
void F();
void E(){
    T();
    while(s[i] ==z{
        i++;
        T();
    }
}
void T(){
    F();
    while(s[i] == '*'){
        i++;
        F();
    }
}
void F(){
    if(s[i] == 'a')
        i++;
    else if(s[i] == '('){
        i++;
        E();
        if(s[i] == ')')
            i++;
        else
            error = 1;
    }
    else
        error = 1;
}
int main(){
    printf("Enter expression: ");
    scanf("%s", s);
    E();
    if(i == strlen(s) && error == 0)
        printf("Accepted");
    else
        printf("Rejected");
    return 0;
}
```

Experiment 11 :

```
#include <stdio.h>

char stack[50], s[50];
int top = -1, i = 0;

char op[] = {'i', '+', '*', '$'};

char table[4][4] = {
    {'=', '>', '>', '>'},
    {'<', '>', '<', '>'},
    {'<', '>', '>', '>'},
    {'<', '<', '<', '='}
};

int pos(char c){
    int j;

    for(j = 0; j < 4; j++)
        if(op[j] == c)
            return j;

    return -1;
}

void display(){
    int j;

    for(j = 0; j <= top; j++)
        printf("%c", stack[j]);

    printf("\t\t");

    for(j = i; s[j] != '\0'; j++)
        printf("%c", s[j]);

    printf("\t\t");
}

void parse(){
    char rel, ch;
    int p1, p2;

    printf("Enter the expression for operator precedence
parsing(end with $): ");
    scanf("%s", s);

    stack[++top] = '$';

    while(1){
        display();

        if(stack[top] == '&&s[i] == ') {
            printf("\nValid Expression\n");
            return;
        }

        p1 = pos(stack[top]);
        p2 = pos(s[i]);

        if(p1 == -1 || p2 == -1){
            printf("\nInvalid expression\n");
            return;
        }

        rel = table[p1][p2];

        if(rel == '<' || rel == '='){
            stack[++top] = s[i++];
            printf("Shift\n");
        }
        else if(rel == '>'){
            ch = stack[top--];

            while(table[pos(stack[top])][pos(ch)] != '<')
                ch = stack[top--];

            printf("%c ", ch);
            printf("Reduce\n");
        }
        else
        {
            printf("\nInvalid expression\n");
            return;
        }
    }
}

int main(){
    parse();
    return 0;
}
```

OUTPUT :

Enter the expression for operator precedence parsing(end with): $i + i * i$

$	i+i*i$	Shift
$i	+i*i$	i Reduce
$	+i*i$	Shift
$+	i*i$	Shift
$+i	*i$	i Reduce
$+	*i$	Shift
$+*	i$	Shift
$+*i	$	i Reduce
$+*	$	* Reduce
$+	$	+ Reduce
$	$	

Valid Expression

Experiment 12 :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

void input();
void output();
void change(int p, char *res);
void constant();

struct expr{
   char op[2], op1[5], op2[5], res[5];
   int flag;
} arr[10];

int n;

int main(){
   input();
   constant();
   output();

   return 0;
}

void input(){
   int i;

   printf("\nEnter the maximum number of expressions:
");
   scanf("%d", &n);

   printf("\nEnter the input:\n");

   for(i = 0; i < n; i++)   {
      scanf("%s %s %s %s",
           arr[i].op,
           arr[i].op1,
           arr[i].op2,
           arr[i].res);

      arr[i].flag = 0;
   }
}

void constant(){
   int i, op1, op2, res;
   char op, res1[5];

   for(i = 0; i < n; i++){
      if((isdigit(arr[i].op1[0]) &&
          isdigit(arr[i].op2[0])) ||
          strcmp(arr[i].op, "=") == 0){
         op1 = atoi(arr[i].op1);
         op2 = atoi(arr[i].op2);
         op = arr[i].op[0];

         switch(op){
            case '+': res = op1 + op2; break;
            case '-': res = op1 - op2; break;
            case '*': res = op1 * op2; break;
            case '/': res = op1 / op2; break;
            case '=': res = op1; break;
         }

         sprintf(res1, "%d", res);

         arr[i].flag = 1;
         change(i, res1);
      }
   }
}

void change(int p, char *res){
   int i;

   for(i = p + 1; i < n; i++){
      if(strcmp(arr[p].res, arr[i].op1) == 0)
         strcpy(arr[i].op1, res);

      else if(strcmp(arr[p].res, arr[i].op2) == 0)
         strcpy(arr[i].op2, res);
   }
}

void output()
{
   int i;

   printf("\nOptimized code is:");

   for(i = 0; i < n; i++)
   {
      if(!arr[i].flag)
         printf("\n%s %s %s %s",
               arr[i].op,
               arr[i].op1,
               arr[i].op2,
               arr[i].res);
   }
}
```

OUTPUT :

```
Enter the maximum number of expressions: 4

Enter the input:
= 5 - a
+ a b t1
* t1 c t2
+ t2 d t3

Optimized code is:
+ 5 b t1
* t1 c t2
+ t2 d t3
```

Experiment 13 :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[20][10];
int top = -1, temp = 1;

void push(char s[]){
   strcpy(stack[++top], s);
}

char *pop(){
   return stack[top--];
}

int prec(char c){
   if(c == '+' || c == '-') return 1;
   if(c == '*' || c == '/') return 2;
   if(c == '^') return 3;
   return 0;
}

int main(){
   char infix[30], postfix[30];
   char a[10], b[10], t[10];
   int i, j = 0;

   printf("Enter expression: ");
   scanf("%s", infix);

   for(i = 0; infix[i]; i++)   {
      if(isalnum(infix[i]))
         postfix[j++] = infix[i];

      else if(infix[i] == '('){
         char x[2] = "(";
         push(x);
      }

      else if(infix[i] == ')'){
    while(strcmp(stack[top], "(") != 0)
            postfix[j++] = pop()[0];

         pop();
      }
      else{
         while(top != -1 && prec(stack[top][0]) >=
prec(infix[i]))
            postfix[j++] = pop()[0];

         char x[2] = {infix[i], '\0'};
         push(x);
      }
   }

   while(top != -1)
      postfix[j++] = pop()[0];

   postfix[j] = '\0';

   printf("Postfix form: %s\n", postfix);
   top = -1;

   printf("\nIntermediate Code:\n");

   for(i = 0; postfix[i]; i++){
      if(isalnum(postfix[i])) {
         char x[2] = {postfix[i], '\0'};
         push(x);
      }
      else  {
         strcpy(b, pop());
         strcpy(a, pop());

         sprintf(t, "t%d", temp);

         printf("%s = %s %c %s\n",
               t, a, postfix[i], b);

         push(t);
         temp++;
      }
   }

   return 0;
}
```

OUTPUT :

```
Enter expression: a+b*c
Postfix form: abc*+

Intermediate Code:
t1 = b * c
t2 = a + t1
```

Experiment 14 :

```
#include <stdio.h>

int main()
{
   int n, i;
   char op[10][2], a[10][10], b[10][10], r[10][10];

   printf("Enter number of instructions: ");
   scanf("%d", &n);

   printf("Enter TAC:\n");

   /* Read all TAC instructions first */
   for(i = 0; i < n; i++)
   {
      scanf("%s %s %s %s", op[i], a[i], b[i], r[i]);
   }

   /* Generate assembly after reading all instructions */
   printf("\n8086 Assembly Code:\n");

   for(i = 0; i < n; i++)
   {
      if(op[i][0] == '+')
      {
         printf("MOV AX,%s\n", a[i]);
         printf("ADD AX,%s\n", b[i]);
         printf("MOV %s,AX\n\n", r[i]);
      }

      else if(op[i][0] == '-')
      {
         printf("MOV AX,%s\n", a[i]);
         printf("SUB AX,%s\n", b[i]);
         printf("MOV %s,AX\n\n", r[i]);
      }

      else if(op[i][0] == '*')
      {
         printf("MOV AX,%s\n", a[i]);
         printf("MOV BX,%s\n", b[i]);
         printf("MUL BX\n");
         printf("MOV %s,AX\n\n", r[i]);
      }

      else if(op[i][0] == '/')
      {
         printf("MOV AX,%s\n", a[i]);
         printf("MOV BX,%s\n", b[i]);
         printf("DIV BX\n");
         printf("MOV %s,AX\n\n", r[i]);
      }
   }

   return 0;
}
```

OUTPUT :

```
Enter number of instructions: 4
Enter TAC:
+ a b t1
- c d t2
* e f t3
/ g h t4

8086 Assembly Code:
MOV AX,a
ADD AX,b
MOV t1,AX

MOV AX,c
SUB AX,d
MOV t2,AX

MOV AX,e
MOV BX,f
MUL BX
MOV t3,AX

MOV AX,g
MOV BX,h
DIV BX
MOV t4,AX
```